

SEGMENT 06 • DEMO

Kivi, Sarvam Code and the tooling layer

Products built on the stack — and how to make your AI assistant good at it

Kivi • Kaze • Sarvam Code • MCP server • Agent Skills • llms.txt • Context7

Dr. Bhaveshkumar C. Dharmani • PhD (ICT), DA-IICT Gandhinagar • Founder, AI Vidhya4 Sarvam

<https://www.aividhya.in/> • <https://www.aividhya4sarvam.in/> • bhavesh@aividhya.in • <https://www.linkedin.com/in/bhaveshdharmani/>

Kivi — the stack, productised



What it is

A desktop voice tool. Type, code and control your computer by speech. Built on Saaras V4, natively multilingual across Indian languages.



The comparison people make

Similar in shape to Whisper Flow and other dictation tools — but pitched around multilingual use and full computer control, not just transcription.



Why a developer should care

This is what Saaras V4 makes possible as a product rather than an API. The category is wide open in every vertical.



What we are doing now

I demo it live — dictating in Hindi, controlling the machine, writing code by voice. You download it tonight and try it.



Why you are watching rather than installing

Getting a hundred people to install desktop software across three operating systems mid-session costs twenty minutes and derails the room. Watch now, install tonight, tell me tomorrow what you built with it.



The interesting question is not 'is Kivi good'. It is: what is the Kivi of your industry, and has anyone built it yet?

Kaze — where this is heading

Kaze

AI smart glasses that understand
what you see

Edge

on-device inference for offline
and low-connectivity

8kHz

telephony — the other end
of the same spectrum

- The interesting thing about Kaze and Edge together is what they imply: Sarvam is not assuming a good internet connection or a modern phone.
- For rural India that is not a nice-to-have. A field officer in a district with patchy 4G, a farmer on a feature phone, a health worker in a village — these are the actual users of population-scale AI.
- Sarvam Edge is the one to watch if your product has to work where connectivity does not. Very few global providers are building for that constraint at all.

B

Make your assistant good at this stack

Four tools, ten minutes of setup, permanent payoff

Why your AI assistant gets Sarvam wrong

1

Training data goes stale

This stack shipped a new flagship LLM, two speech generations, a vision upgrade and a rebuilt Document AI in six months. Any model's priors are out of date.

2

Method names break OpenAI convention

`client.chat.completions(...)` — not `.create(...)`. Every assistant writes `.create()` by default, because that is what every other OpenAI-shaped client does.

3

Parameters fail silently

`output_script` ignored on `sarvam-translate`. `language_code` ignored on Document AI. HTTP 200, wrong output, no warning.

4

Response fields behave unexpectedly

`content` can be `None` when reasoning consumes the token budget. An assistant that has not seen this writes code that crashes on a null.



None of these are hard once you know them. All of them cost you an afternoon the first time. That is precisely the gap Agent Skills fill.

The install — thirty seconds

BASH

```
npx skills add sarvamai/skills           # everything
npx skills add sarvamai/skills --skill chat # just one
npx skills add sarvamai/skills --list      # browse first

export SARVAM_API_KEY="your-key"  &&  pip install sarvamai
```

Skill	What it corrects
chat	Sarvam-105B/30B: quick starts, streaming, reasoning mode, the OpenAI-compatible path, the content=None gotcha
speech-to-text	Saaras v3: quick starts, Batch API for long audio + diarization, WebSocket streaming across 23 languages
text-to-speech	Bulbul v3: HTTP streaming, WebSocket, pronunciation dictionaries, unsupported-parameter warnings
translate	Sarvam-Translate v1 and Mayura v1: feature differences and silent parameter failures
voice-agents	Real-time agents with LiveKit and Pipecat (Python), plus a JS/TS reference for custom pipelines
vibe-coding	Vendor-neutral habits: small steps, verification, skills over long prompts. Pair with a domain skill



Works with anything supporting the Agent Skills spec — Claude Code, Cursor, Windsurf. Install once, and every future session reads them automatically.

The proof — do this live



Without skills installed

- Ask: "write a Saaras transcription script"
- `client.chat.completions.create(...)`
- → `AttributeError`
- Invents parameters that do not exist
- Uses `saarika:v2.5` — the legacy model
- Misses `mode=` entirely
- Writes a plausible script that does not run



With skills installed

- Same prompt, same assistant
- `client.chat.completions(...)`
- → works
- Correct current model IDs
- Knows the five modes exist
- Handles `content=None`
- Links out to `llms.txt` for parameter tables



Ninety seconds, and it lands harder than any explanation. Ask someone in the room to run it too, so it is not just my machine.

Three ways to get Sarvam knowledge into an assistant

	Agent Skills	MCP server	llms.txt
When it loads	Once, installed into your project or editor	Live, queried on demand while you code	Fetches manually when you need it
Best for	Baking in SDK corrections an assistant keeps getting wrong	Interactive sessions needing current API details, or calling Sarvam directly	Bulk ingestion, RAG indexes, one-shot context
Setup	<code>npx skills add sarvamai/skills</code>	One-time MCP client config	None — just a URL
Gives you	Corrections	Live tool access + docs	The whole corpus



Use all three. A skill stops the wrong method name. The MCP server and llms.txt supply the parameter tables a skill deliberately leaves out.

llms.txt — the trick nobody uses

APPEND .md TO ANY DOCS PAGE. OR TAKE THE WHOLE CORPUS.

```
# Markdown version of any page  
https://docs.sarvam.ai/api/getting-started/models.md
```

```
# Page-level index  
https://docs.sarvam.ai/api/getting-started/llms.txt
```

```
# Full index of the documentation  
https://docs.sarvam.ai/llms.txt
```

```
# THE ENTIRE DOCUMENTATION IN ONE FILE  
https://docs.sarvam.ai/llms-full.txt
```

1

Paste llms-full.txt into your assistant's context

Instant, current, complete knowledge of the whole API surface. No RAG pipeline, no embedding step, no staleness.

2

Or index it for a real RAG setup

It is structured markdown, designed for ingestion. This is what you build a documentation assistant on top of.

3

Also: Context7 and the docs MCP server

docs.sarvam.ai/_mcp/server connects your editor straight to the documentation. Different from the API MCP server — this one is for reading, that one is for doing.

Sarvam Code and the rest of the toolkit



Sarvam Code (Beta)

A coding assistant built on open models, for writing and understanding code. Announced at Epoch.



API Playground

dashboard.sarvam.ai/playground — test every endpoint in the browser with no setup. Where you send someone who is stuck.



The Cookbook

github.com/sarvamai/sarvam-ai-cookbook — working notebooks for STT, TTS, translation, doc intelligence, voice agents.



Official SDKs

`pip install sarvamai` • `npm install sarvamai`. Async, retries and streaming built in. Typed clients.



AI docs assistant

Built into the documentation search bar. Fastest route to an answer when you are mid-build.



Status and changelog

status.sarvam.ai for incidents. The changelog is the one page to check before every project — this stack moves.



Contributing a cookbook example is the cheapest developer marketing available to you — and it puts your name in front of the team.

The new cook and the recipe card taped to the wall

You can explain the kitchen's quirks to every new cook, every shift, out loud. The left burner runs hot. The oven is twenty degrees off. Salt goes in at the end, not the start.

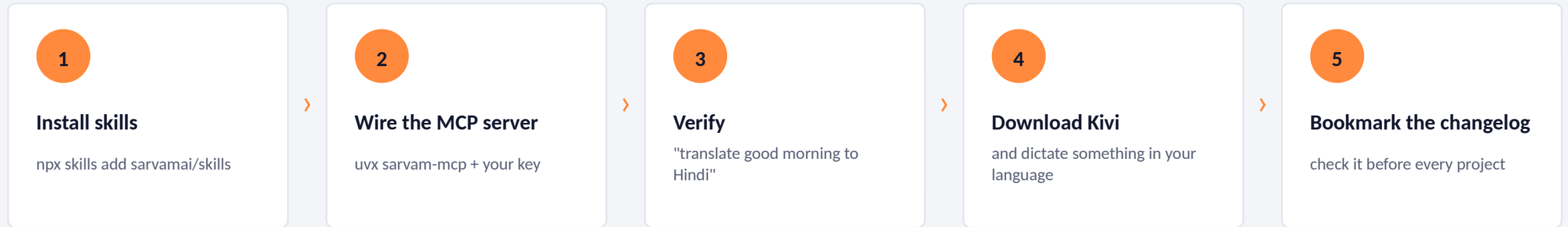
Or you can tape a card to the wall.

The card does not make anyone a better cook. It just stops the same four mistakes being made forever.



That is what an Agent Skill is. Not intelligence — institutional memory. Install it once and stop re-explaining that the method is `client.chat.completions()`, not `.create()`.

Set this up tonight — ten minutes



★ **Do this before you write another line of Sarvam code. Ten minutes tonight saves you an afternoon next week — and I would rather you spent that afternoon building.**

→ **Next: the money segment**

Everything so far has been capability. Now we find out what it costs, what you can charge, and where the business actually is. This is the twenty-five minutes people repeat to colleagues on Monday.

“

Institutional memory beats intelligence.
Tape the card to the wall.

— End of Segment 06. Next: unit economics.